SciPy Introduction

SciPy stands for Scientific Python is an open-source Python library used for scientific computing, mathematics, engineering, optimization, statistics, signal processing, image processing, and numerical integration.

SciPy is built on top of **NumPy** and provides many advanced mathematical functions.

SciPy was created by NumPy's creator Travis Olliphant.

Why Use SciPy?

If SciPy uses NumPy underneath, why can we not just use NumPy?

SciPy has optimized and added functions that are frequently used in NumPy and Data Science.

Which Language is SciPy Written in?

SciPy is predominantly written in Python, but a few segments are written in C.

Where is the SciPy Codebase?

The source code for SciPy is located at this github repository
https://github.com/scipy/scipy

Github: enables many people to work on the same codebase.

Installation of SciPy

If you have Python and PIP already installed on a system, then installation of SciPy is very easy.

Install it using this command: `pip install scipy`

If this command fails, then use a Python distribution that already has SciPy installed like, Anaconda, and Spyder etc.

Import SciPy

Once SciPy is installed, import the SciPy module(s) you want to use in your applications by adding the `from scipy import module` statement:

```
from scipy import constants
```

Now we have imported the *constants* module from SciPy, and the application is ready to use it:

Example:

How many cubic meters are in one liter?

```
from scipy import constants

print(constants.liter)      # 0.001
```

Checking SciPy Version

The version string is stored under the `__version__` attribute.

Example:

```
import scipy

print(scipy.__version__)    # 1.16.3
```

SciPy Constants

Constants in SciPy

As SciPy is more focused on scientific implementations, it provides many built-in scientific constants.

These constants can be helpful when you are working with Data Science.

PI is an example of a scientific constant.

Example:

Print the constant value of PI:

```
from scipy import constants

print(constants.pi)    # 3.141592653589793
```

Constant Units

A list of all units under the constants module can be seen using the `dir()` function.

Example:

List all constants:

```
from scipy import constants

print(dir(constants))
```

Output:
```
['Avogadro', 'Boltzmann', 'Btu', 'Btu_IT', 'Btu_th',
'ConstantWarning', 'G', 'Julian_year', 'N_A', 'Planck', 'R', 'Rydberg',
'Stefan_Boltzmann', 'Wien', '__all__', '__builtins__', '__cached__',
'__doc__', '__file__', '__loader__', '__name__', '__package__',
'__path__', '__spec__', '_codata', '_constants', '_obsolete_constants',
'acre', 'alpha', 'angstrom', 'arcmin', 'arcminute', 'arcsec',
'arcsecond', 'astronomical_unit', 'atm', 'atmosphere', 'atomic_mass',
'atto', 'au', 'bar', 'barrel', 'bbl', 'blob', 'c', 'calorie',
'calorie_IT', 'calorie_th', 'carat', 'centi', 'codata', 'constants',
'convert_temperature', 'day', 'deci', 'degree', 'degree_Fahrenheit',
'deka', 'dyn', 'dyne', 'e', 'eV', 'electron_mass', 'electron_volt',
'elementary_charge', 'epsilon_0', 'erg', 'exa', 'exbi', 'femto',
'fermi', 'find', 'fine_structure', 'fluid_ounce', 'fluid_ounce_US',
'fluid_ounce_imp', 'foot', 'g', 'gallon', 'gallon_US', 'gallon_imp',
'gas_constant', 'gibi', 'giga', 'golden', 'golden_ratio', 'grain',
'gram', 'gravitational_constant', 'h', 'hbar', 'hectare', 'hecto',
```

```
'horsepower', 'hour', 'hp', 'inch', 'k', 'kgf', 'kibi', 'kilo',
'kilogram_force', 'kmh', 'knot', 'lambda2nu', 'lb', 'lbf',
'light_year', 'liter', 'litre', 'long_ton', 'm_e', 'm_n', 'm_p', 'm_u',
'mach', 'mebi', 'mega', 'metric_ton', 'micro', 'micron', 'mil', 'mile',
'milli', 'minute', 'mmHg', 'mph', 'mu_0', 'nano', 'nautical_mile',
'neutron_mass', 'nu2lambda', 'ounce', 'oz', 'parsec', 'pebi', 'peta',
'physical_constants', 'pi', 'pico', 'point', 'pound', 'pound_force',
'precision', 'proton_mass', 'psi', 'pt', 'quecto', 'quetta', 'ronna',
'ronto', 'short_ton', 'sigma', 'slinch', 'slug', 'speed_of_light',
'speed_of_sound', 'stone', 'survey_foot', 'survey_mile', 'tebi',
'tera', 'test', 'ton_TNT', 'torr', 'troy_ounce', 'troy_pound', 'u',
'unit', 'value', 'week', 'yard', 'year', 'yobi', 'yocto', 'yotta',
'zebi', 'zepto', 'zero_Celsius', 'zetta']
```

Unit Categories

The units are placed under these categories:

- Metric
- Binary
- Mass
- Angle
- Time
- Length
- Pressure
- Volume
- Speed
- Temperature
- Energy
- Power
- Force

Metric (SI) Prefixes:

Return the specified unit in **meter** (e.g. `centi` returns `0.01`)

Example:

```
from scipy import constants

print(constants.yotta)    #1e+24
print(constants.zetta)    #1e+21
print(constants.exa)      #1e+18
print(constants.peta)     #1000000000000000.0
print(constants.tera)     #1000000000000.0
```

```
print(constants.giga)     #1000000000.0
print(constants.mega)     #1000000.0
print(constants.kilo)     #1000.0
print(constants.hecto)    #100.0
print(constants.deka)     #10.0
print(constants.deci)     #0.1
print(constants.centi)    #0.01
print(constants.milli)    #0.001
print(constants.micro)    #1e-06
print(constants.nano)     #1e-09
print(constants.pico)     #1e-12
print(constants.femto)    #1e-15
print(constants.atto)     #1e-18
print(constants.zepto)    #1e-21
```

Binary Prefixes:

Return the specified unit in **bytes** (e.g. `kibi` returns `1024`)

```
from scipy import constants

print(constants.kibi)    #1024
print(constants.mebi)    #1048576
print(constants.gibi)    #1073741824
print(constants.tebi)    #1099511627776
print(constants.pebi)    #1125899906842624
print(constants.exbi)    #1152921504606846976
print(constants.zebi)    #1180591620717411303424
print(constants.yobi)    #1208925819614629174706176
```

Mass:

Return the specified unit in **kg** (e.g. `gram` returns `0.001`)

```
from scipy import constants

print(constants.gram)         #0.001
print(constants.metric_ton)   #1000.0
print(constants.grain)        #6.479891e-05
print(constants.lb)           #0.45359236999999997
print(constants.pound)        #0.45359236999999997
print(constants.oz)           #0.028349523124999998
print(constants.ounce)        #0.028349523124999998
print(constants.stone)        #6.3502931799999995
```

```
print(constants.long_ton)     #1016.0469088
print(constants.short_ton)    #907.1847399999999
print(constants.troy_ounce)   #0.031103476799999998
print(constants.troy_pound)   #0.37324172159999996
print(constants.carat)        #0.0002
print(constants.atomic_mass) #1.66053904e-27
print(constants.m_u)          #1.66053904e-27
print(constants.u)            #1.66053904e-27
```

Angle:

Return the specified unit in **radians** (e.g. `degree` returns `0.017453292519943295`)

Example:

```
from scipy import constants

print(constants.degree)     #0.017453292519943295
print(constants.arcmin)     #0.0002908882086657216
print(constants.arcminute)  #0.0002908882086657216
print(constants.arcsec)     #4.84813681109536e-06
print(constants.arcsecond)  #4.84813681109536e-06
```

Time:

Return the specified unit in **seconds** (e.g. `hour` returns `3600.0`)

Example:

```
from scipy import constants

print(constants.minute)      #60.0
print(constants.hour)        #3600.0
print(constants.day)         #86400.0
print(constants.week)        #604800.0
print(constants.year)        #31536000.0
print(constants.Julian_year) #31557600.0
```

Length:

Return the specified unit in **meters** (e.g. `nautical_mile` returns `1852.0`)

```
from scipy import constants

print(constants.inch)              #0.0254
print(constants.foot)              #0.30479999999999996
print(constants.yard)              #0.9143999999999999
print(constants.mile)              #1609.3439999999998
print(constants.mil)               #2.5399999999999997e-05
print(constants.pt)                #0.00035277777777777776
print(constants.point)             #0.00035277777777777776
print(constants.survey_foot)       #0.3048006096012192
print(constants.survey_mile)       #1609.3472186944373
print(constants.nautical_mile)     #1852.0
print(constants.fermi)             #1e-15
print(constants.angstrom)          #1e-10
print(constants.micron)            #1e-06
print(constants.au)                #149597870691.0
print(constants.astronomical_unit) #149597870691.0
print(constants.light_year)        #9460730472580800.0
print(constants.parsec)            #3.0856775813057292e+16
```

Pressure:

Return the specified unit in **pascals** (e.g. `psi` returns `6894.757293168361`)

Example:

```
from scipy import constants

print(constants.atm)         #101325.0
print(constants.atmosphere)  #101325.0
print(constants.bar)         #100000.0
print(constants.torr)        #133.32236842105263
print(constants.mmHg)        #133.32236842105263
print(constants.psi)         #6894.757293168361
```

Area:

Return the specified unit in **square meters**(e.g. `hectare` returns `10000.0`)

Example:

```
from scipy import constants

print(constants.hectare) #10000.0
print(constants.acre)    #4046.8564223999992
```

Volume:

Return the specified unit in **cubic meters** (e.g. `liter` returns `0.001`)

Example:

```
from scipy import constants

print(constants.liter)            #0.001
print(constants.litre)            #0.001
print(constants.gallon)           #0.003785411783999997
print(constants.gallon_US)        #0.003785411783999997
print(constants.gallon_imp)       #0.00454609
print(constants.fluid_ounce)      #2.9573529562499998e-05
print(constants.fluid_ounce_US)   #2.9573529562499998e-05
print(constants.fluid_ounce_imp)  #2.84130625e-05
print(constants.barrel)           #0.15898729492799998
print(constants.bbl)              #0.15898729492799998
```

Speed:

Return the specified unit in **meters per second** (e.g. `speed_of_sound` returns `340.5`)

Example:

```
from scipy import constants

print(constants.kmh)            #0.2777777777777778
print(constants.mph)            #0.44703999999999994
print(constants.mach)           #340.5
print(constants.speed_of_sound) #340.5
print(constants.knot)           #0.5144444444444445
```

Temperature:

Return the specified unit in **Kelvin** (e.g. `zero_Celsius` returns `273.15`)

Example:

```
from scipy import constants

print(constants.zero_Celsius)      #273.15
print(constants.degree_Fahrenheit) #0.5555555555555556
```

Energy:

Return the specified unit in **joules** (e.g. `calorie` returns `4.184`)

Example:

```
from scipy import constants

print(constants.eV)            #1.6021766208e-19
print(constants.electron_volt) #1.6021766208e-19
print(constants.calorie)       #4.184
print(constants.calorie_th)    #4.184
print(constants.calorie_IT)    #4.1868
print(constants.erg)           #1e-07
print(constants.Btu)           #1055.05585262
print(constants.Btu_IT)        #1055.05585262
print(constants.Btu_th)        #1054.3502644888888
print(constants.ton_TNT)       #4184000000.0
```

Power:

Return the specified unit in **watts** (e.g. `horsepower` returns `745.6998715822701`)

Example:

```
from scipy import constants

print(constants.hp)         #745.6998715822701
print(constants.horsepower) #745.6998715822701
```

Force:

Return the specified unit in **newton** (e.g. `kilogram_force` returns `9.80665`)

Example:

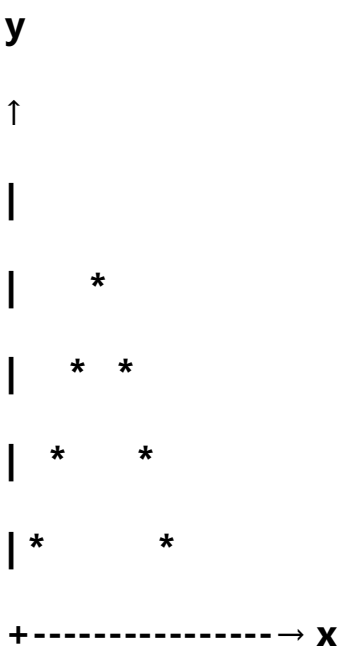
```
from scipy import constants

print(constants.dyn)             #1e-05
print(constants.dyne)            #1e-05
print(constants.lbf)             #4.4482216152605
print(constants.pound_force)     #4.4482216152605
print(constants.kgf)             #9.80665
print(constants.kilogram_force)  #9.80665
```

Numerical Integration

Integration is the process of finding the **area under a curve**.

Suppose we have a graph:



The shaded area under the curve can be calculated using integration.

In mathematics, integration is represented as:

$$\int_a^b f(x)dx$$

Where:

- f(x) = function
- a = starting point
- b = ending point
- dx = very small change in x

Why Do We Need Numerical Integration?

For simple functions:

$$\int x\text{^}2dx$$

We can solve manually.

However, many real-world problems involve:

- Complex equations
- Experimental data
- Sensor measurements
- Weather observations
- Engineering calculations

For such cases, analytical integration may be difficult or impossible.

Numerical Integration provides an approximate answer using computers.

Real-Life Applications

- **Area of a Land:** A farmer wants to know the area of an irregular field.
- **Distance Travelled:** If speed varies with time, Distance is obtained using integration.
- **Water Flow:** Engineers calculate the amount of water flowing through a pipe.
- **Population Growth:** Scientists calculate total population change over time.

Single Integration using quad()

This is the most commonly used integration function.

Syntax:

```
from scipy.integrate import quad

result, error = quad(function, lower_limit, upper_limit)
```

Example:

Find: $\int_0^1 x^2 dx$

Mathematical Solution

$$\left[\frac{x^3}{3}\right]_0^1$$

$$= \frac{1}{3}$$

SciPy Solution

```
from scipy.integrate import quad

def f(x):
    return x**2

result, error = quad(f, 0, 1)

print("Result =", result)
print("Error =", error)
```

Output:

```
Result = 0.33333333333333337
Error = 3.700743415417189e-15
```

Integrating Trigonometric Functions

Find: $\int_0^{\Pi} Sin(x) dx$

Solution:

```
from scipy.integrate import quad
import numpy as np

result, error = quad(np.sin, 0, np.pi)

print(result) # 2.0
```

Integrating Exponential Functions

Find: $\int_0^5 e^{-x}\,dx$

Solution:

```
from scipy.integrate import quad
import numpy as np

result, error = quad(
    lambda x: np.exp(-x),
    0,
    5
)

print(result) # 0.9932620530009145
```

Infinite Integrals

SciPy can even integrate to infinity.

Find: $\int_0^\infty e^{-x}\,dx$

Solution:

```
from scipy.integrate import quad
import numpy as np

result, error = quad(
    lambda x: np.exp(-x),
    0,
    np.inf
)

print(result) # 1.0
```

Understanding the Error Value

When using:

```
result, error = quad(...)
```

SciPy returns:

- Result = estimated integral
- Error = estimation error

Example:

```
0.333333333333(R)
3.7e-15(E)
```

The error is extremely small.

Hence the result is highly accurate.

Double Integration

Used when a function depends on two variables.

Example:

$$\int_0^1 \int_0^1 xy \, dy \, dx$$

Solution:

```
from scipy.integrate import dblquad

f = lambda y, x: x*y

result, error = dblquad(
    f,
    0, 1,
    lambda x: 0,
    lambda x: 1
)

print(result)     #0.25
```

Student Activity

1. **Calculate $\int_0^1 x^3 dx$ using quad()**
2. **Calculate $\int_0^{\Pi} cos(x) dx$**